

Visualização de Dados com Matplotlib

Como transformar dados brutos de mercado

em insights visuais poderosos

usando Python, Matplotlib e a API da Binance.

PYTHON

MATPLOTLIB

BINANCE

DADOS FINANCEIROS



Tarsis Lima (@tarsislimadev)

Entusiasta de dados e desenvolvedor apaixonado

Experiência em Java e Node.js

Aprendendo Python, finanças e visualização de dados

Interesse em mercado financeiro e trading algorítmico

<https://x.com/tarsislimadev>

<https://github.com/tarsislimadev>

<https://www.linkedin.com/in/tarsislimadev/>



O Desafio: De Dados Brutos a Insights

O mercado financeiro gera volumes massivos de dados a cada segundo — preços, volumes, ordens e indicadores se acumulam em fluxos contínuos que são impossíveis de interpretar no formato numérico bruto. Números isolados, sem contexto visual, tornam a tomada de decisão lenta e propensa a erros.

Volume Massivo

A Binance processa milhões de transações por dia, gerando um fluxo contínuo de dados que exige ferramentas adequadas para interpretação.

Complexidade Numérica

Tabelas com milhares de linhas de preços e volumes são pouco intuitivas — o cérebro humano processa padrões visuais com muito mais eficiência.

Matplotlib como Solução

A biblioteca transforma essa complexidade em clareza, permitindo criar gráficos que revelam tendências, padrões e anomalias em segundos.

O que é Matplotlib?

Definição

Matplotlib é a biblioteca de visualização de dados mais consolidada do ecossistema Python. Criada em 2003 por John Hunter, tornou-se o padrão da indústria para geração de gráficos em ciência de dados, engenharia e finanças.

É mantida ativamente pela comunidade open-source e serve como base para bibliotecas mais modernas como Seaborn e Plotly.

Principais Características



Gráficos Estáticos: Geração de imagens em alta resolução para relatórios e publicações.



Animações: Atualização dinâmica de gráficos em tempo real, ideal para dashboards ao vivo.



Estrutura Fig/Axes: Controle granular sobre cada elemento visual — cores, fontes, escalas e layouts.



Multiplataforma: Funciona em Jupyter, scripts .py, interfaces Tkinter e servidores web.

Exemplo Prático: Plotando Séries Temporais

O fluxo básico de trabalho com Matplotlib segue quatro passos essenciais. Veja como criar seu primeiro gráfico de série temporal:

Código Base

```
import matplotlib.pyplot as plt
import numpy as np

# Dados de exemplo
dias = np.arange(1, 11)
precos = [100, 102, 98, 105, 107, 103, 110, 112, 108, 115]

# Criar figura e eixos
fig, ax = plt.subplots(figsize=(10, 6))

# Plotar série temporal
ax.plot(dias, precos, marker='o', linewidth=2)

# Customizar
ax.set_title('Evolução de Preço')
ax.set_xlabel('Dia')
ax.set_ylabel('Preço (USD)')
ax.grid(True, alpha=0.3)

plt.show()
```

Entendendo Cada Etapa

01

Importação

`import matplotlib.pyplot as plt` – importa o módulo principal com o alias padrão da comunidade.

02

Configuração

`fig, ax = plt.subplots()` – cria a figura e os eixos simultaneamente, pronto para plotar.

03

Plotagem

`ax.plot(x, y)` – desenha a linha conectando os pontos. Aceita marcadores, cores e estilos.

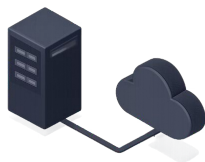
04

Exibição

`plt.show()` – renderiza e exibe a janela com o gráfico finalizado.

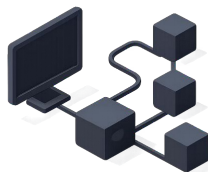
Integração: Conectando com a Binance

Para visualizar dados reais do mercado de criptomoedas, o projeto se conecta à Binance usando dois protocolos complementares que garantem tanto o contexto histórico quanto a atualização em tempo real.



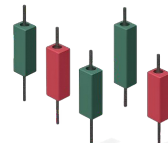
REST API — Histórico Inicial

A API REST da Binance fornece dados de velas (kline) históricos. O projeto faz uma requisição inicial para capturar o passado recente — os últimos 500 (quinhentos) candles — criando o contexto necessário antes de iniciar a visualização ao vivo.



WebSocket — Dados em Tempo Real

Após carregar o histórico, o projeto se conecta ao stream WebSocket da Binance. Esse canal mantém uma conexão persistente que empurra novos candles segundo a segundo, sem necessidade de web polling, garantindo latência mínima.



Dados Kline (Candlestick)

Cada candle contém: preço de abertura, máxima, mínima, fechamento, e volume. Esses cinco valores são a base para construir gráficos de velas japonesas, indicadores técnicos e análises de tendência.

Visualização Dinâmica em Ação

Como Funciona o Live Plot

O core do projeto é o loop de atualização dinâmica. A cada novo candle recebido via WebSocket, o Matplotlib redesenha o gráfico sem fechar a janela, criando a sensação de um terminal de trading profissional.

Atualização Contínua

O gráfico é redesenhado com `plt.pause()` ou `FuncAnimation`, mantendo a janela responsiva enquanto novos dados chegam.

Múltiplos Eixos

Preço e volume são plotados em eixos Y separados (secundário), permitindo comparar as duas dimensões sem distorção de escala.

Estilo de Candles

Candles de alta **em verde** e de baixa **em vermelho** — convenção universal do mercado financeiro, facilmente customizável no Matplotlib.



Por que Matplotlib?

Flexibilidade Total

Controle absoluto sobre cada pixel do gráfico — de fontes a grades, de cores a anotações. Ideal para gráficos de qualidade de publicação acadêmica ou relatórios profissionais.

Suporte Nativo ao Jupyter

Integração perfeita com Jupyter Notebooks e JupyterLab, permitindo visualização inline durante a exploração de dados e análise interativa.

Ecossistema Rico

Compatível com Python NumPy, Pandas, Scikit-learn e outras ferramentas de análise. Funciona como base para Seaborn, Plotly e bibliotecas de alto nível.

Comunidade Ativa

Documentação extensa, milhares de exemplos no Stack Overflow e uma comunidade global que mantém a biblioteca atualizada e segura.



Dicas para uma Visualização Eficaz

Um gráfico bem construído comunica mais em segundos do que uma tabela em minutos. Aplique estas boas práticas para elevar a qualidade das suas visualizações financeiras.



Histogramas para Volatilidade

Use `ax.hist()` para visualizar a distribuição de retornos ou variações de preço. Histogramas revelam padrões de volatilidade que gráficos de linha escondem.



Figsizes Proporcional

Defina `figsize=(12, 6)` ou proporções similares para garantir que o gráfico seja legível em diferentes telas e ao ser exportado como imagem.



Design Limpo

Remova elementos desnecessários: grades excessivas, legendas redundantes e cores conflitantes.



Dica Pro: Use `ax.set_facecolor('#1B1B27')` para criar gráficos com tema escuro, reduzindo o cansaço visual em sessões longas de análise. Combine com cores neon para os candles para um visual profissional.

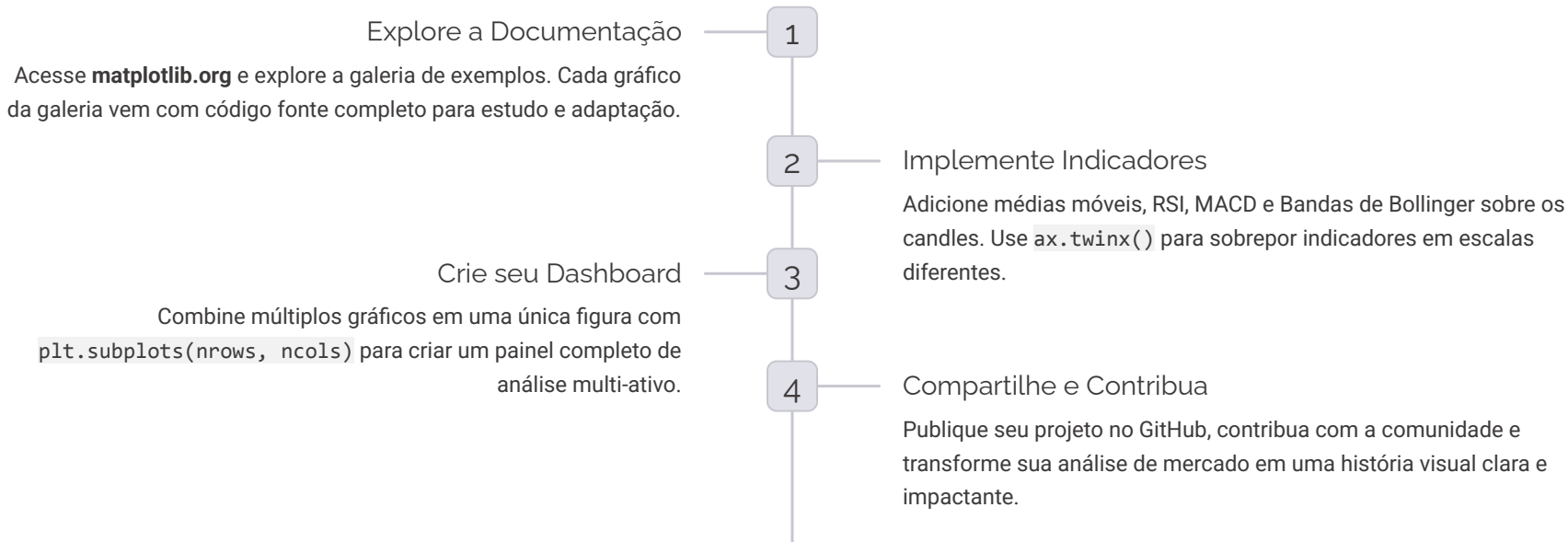


Exportação: Salve seus gráficos com `plt.savefig('grafico.png', dpi=300)` para obter imagens em alta resolução prontas para apresentações e relatórios.

Lembre-se: o protagonista é o dado, não a decoração.

Próximos Passos: Transforme Dados em Estratégia

Você já tem as bases para criar visualizações profissionais. O caminho a seguir é construir sua própria ferramenta de análise de mercado.



🟢 🚀 **Comece hoje:** Clone o repositório `tarsislimadev/fatec-lp2-matplotlib` no GitHub, execute o exemplo e comece a modificar os parâmetros para entender cada componente na prática.